# Markdown: Eine Projekt-README erstellen

Lernmaterial: CC BY 4.0 (siehe `LICENSE-CONTENT.txt`).

## Lernziel

Du kannst eine README-Datei mit Markdown strukturieren und so anpassen, dass andere den Zweck eines Projekts, seine Voraussetzungen und den Einstieg verstehen.

## Was ist Markdown?

Markdown ist eine einfache Auszeichnungssprache für strukturierten Text. Du schreibst Überschriften, Listen, Links und Codeblöcke mit gewöhnlichen Textzeichen. GitHub stellt eine Markdown-Datei wie `README.md` als formatierte Seite dar.

Eine README ist die Einstiegsseite eines Projekts. Sie soll Fragen beantworten, die neue Leserinnen und Leser wahrscheinlich haben: Worum geht es? Was wird benötigt? Wie kann ich beginnen? Welche Dateien oder Bereiche sind wichtig?

## Die Vorlage ansehen

Öffne im Kurs-Repository die Datei `README-Projektvorlage.md`. Sie beschreibt ein fiktives Projekt. Behandle die Angaben deshalb als Beispieldaten, nicht als Tatsachen über dein eigenes Repository.

Markiere in der Vorlage den Titel, die Abschnitte, die Listen und die Stelle für einen eigenen Link. Öffne die Datei einmal als Text und einmal in der gerenderten GitHub-Ansicht. So siehst du, welche Markdown-Zeichen welche Darstellung erzeugen.

Die Vorlage hat absichtlich noch keinen Projektlink: Der passende Link muss auf eine Datei oder einen Ordner in deinem eigenen Lern-Repository zeigen.

## Markdown-Bausteine

### Überschriften

Eine oder mehrere Rauten am Zeilenanfang kennzeichnen eine Überschrift. Eine Raute steht für die oberste Ebene; weitere Rauten bilden untergeordnete Ebenen. Beginne mit einem Projekttitel und ordne die Abschnitte darunter ein.

```
# Projektname

## Zweck

### Einzelheit
```

Lass zwischen Abschnitten eine Leerzeile. Wähle Überschriften nach ihrer Bedeutung – nicht bloß danach, wie groß sie auf dem Bildschirm wirken. GitHub rendert Markdown mit GitHub Flavored Markdown (GFM), einer Variante mit zusätzlichen Funktionen wie Tabellen.[1]

### Absätze und Listen

Eine Leerzeile trennt Absätze. Für eine ungeordnete Liste setzt du einen Bindestrich und ein Leerzeichen vor jeden Punkt:

```
- Ein erster Punkt
- Ein weiterer Punkt
```

Für eine nummerierte Schrittfolge verwendest du Zahlen mit Punkt:

```
1. Voraussetzungen prüfen
2. Projektdateien öffnen
3. Ergebnis kontrollieren
```

[1] GitHub-Dokumentation „Writing on GitHub“, GFM-Spezifikation 0.29-gfm (2019-04-06) und Markdown Guide „Basic Syntax“. Geprüft am 7. November 2026: https://docs.github.com/en/get-started/writing-on-github[2] , https://github.github.com/gfm/[3] , https://www.markdownguide.org/basic-syntax/[4]

### Links

Ein Link besteht aus einem beschreibenden Linktext in eckigen Klammern und einer Zieladresse in runden Klammern:

```
[Projektübersicht](README.md)
```

Der Linktext sollte auch ohne Anklicken verraten, wohin der Link führt. Bei relativen Links gilt die einfache Regel: Prüfe, dass die Zieldatei tatsächlich im Repository liegt.

### Code und Codeblöcke

Kurze Dateinamen oder Befehle im Fließtext setzt du zwischen einzelne Backticks, zum Beispiel `README.md`. Mehrzeilige Befehle und Quellcode kommen zwischen drei Backticks. Eine Sprachangabe wie `bash` direkt nach den öffnenden Backticks kann die Darstellung hervorheben:

````
```bash
printf '%s\n' "Hallo"
```
````

Dokumentiere nur Befehle, die du geprüft hast. Erkläre, wo sie ausgeführt werden und was sie bewirken. Wenn ein Befehl Dateien verändert oder löscht, sag das ausdrücklich dazu.

### Tabellen und Sonderzeichen

Markdown-Tabellen verwenden Pipe-Zeichen (`|`) als Spaltentrenner. Ein Pipe-Zeichen als normaler Text in einer Tabellenzelle muss maskiert werden. Das kann je nach Darstellung fehleranfällig sein.

Für eine kurze Gegenüberstellung ist eine Liste oft einfacher:

- **Pipe (`|`):** leitet die Ausgabe eines Befehls an den nächsten weiter.
- **Umleitung (`>`):** schreibt Ausgabe in eine Datei und kann vorhandenen Inhalt überschreiben.

Tabellen lohnen sich, wenn viele Einträge in denselben Spalten stehen und die Zellen keine Pipe-Zeichen enthalten. Für die meisten README-Dateien reichen Überschriften und Listen.

## Eine README planen

Welche Abschnitte sinnvoll sind, hängt vom Projekt und seiner Zielgruppe ab. Mögliche Bausteine:

- **Projektname und Kurzbeschreibung:** Was ist das Projekt, und wozu dient es?
- **Voraussetzungen:** Welche Programme, Konten oder Daten werden wirklich benötigt?
- **Einstieg oder Installation:** Welche geprüften Schritte führen zum ersten Ergebnis?
- **Nutzung:** Wie wird das Projekt verwendet? Nenne nur Befehle, die es tatsächlich gibt.
- **Dateien und Ordner:** Wo liegen wichtige Bestandteile?
- **Prüfen und Fehlerhilfe:** Woran erkennst du ein korrektes Ergebnis, und wo beginnt die Fehlersuche?
- **Lizenz oder Mitwirkung:** Führe diese Punkte nur auf, wenn die Angaben bekannt und zutreffend sind.

Nicht jedes Projekt braucht jeden Abschnitt. Wenn eine Information noch fehlt, lass sie weg oder kennzeichne sie als offen. Erfinde keine Voraussetzungen oder Befehle.

## Übung: README für dein Lern-Repository

### Voraussetzung

Richte zuerst dein Lern-Repository nach Teil 1 ein. Prüfe im Standardbranch, ob die dort angelegte Workflow-Datei vorhanden ist:

```
.github/workflows/01-hallo-linux.yml
```

Falls sie fehlt, führe zuerst den entsprechenden Schritt aus Teil 1 aus. Verlinke in deiner README nur Dateien, die tatsächlich in deinem Lern-Repository vorhanden sind. Übernimm keine Beispieldaten aus der Vorlage, wenn sie nicht auf dein Repository zutreffen.

### Aufgabe

Erstelle oder ändere die Datei `README.md` in deinem **eigenen Lern-Repository**. Bearbeite für diese Übung nicht die README des Kurs-Repositorys.

Deine README braucht:

1. einen aussagekräftigen Titel,
2. einen kurzen Absatz zum Zweck des Lern-Repositorys,
3. mindestens drei Bereiche oder Dateien aus dem Kurs,
4. Voraussetzungen, die für dein Repository tatsächlich gelten,
5. einen Abschnitt **„Erste Schritte“** mit den nächsten Kursaktionen,
6. mindestens einen funktionierenden Link auf eine Datei oder einen Ordner in deinem Repository.

Nutze `README-Projektvorlage.md` als Orientierung, passe sie an und entferne unzutreffende Beispieldaten. Schreibe keine Installationsbefehle oder Testergebnisse hinein, die du nicht geprüft hast.

### Ausführung

Öffne dein Lern-Repository auf GitHub und bearbeite dort `README.md` im Browser. Schreibe den Inhalt mit Markdown-Syntax und nutze **Preview**, bevor du speicherst. Vergleiche die Vorschau mit dem Quelltext – besonders Überschriften, Listen und Links.

Falls du bereits einen Arbeitsbranch angelegt hast, wähle ihn aus. Andernfalls kannst du diese erste README-Änderung auf dem Standardbranch speichern. Gib eine kurze Commit-Nachricht ein, die die Änderung beschreibt, und bestätige **Commit changes**. Wenn GitHub anbietet, die Änderung auf einem neuen Branch zu speichern, prüfe die Branch-Auswahl.

Der Ablauf entspricht dem Abschnitt „Commits und Branches“ in den Git-Grundlagen.

### Ergebniskontrolle

Prüfe nach dem Speichern:

- Der Titel erscheint als Hauptüberschrift.
- Abschnitte stehen in der richtigen Reihenfolge und Hierarchie.
- Listenpunkte sind gut lesbar.
- Der Link führt zu einer vorhandenen Datei oder einem vorhandenen Ordner.
- Die README enthält keine unzutreffenden Beispieldaten oder ungeprüften Befehle.
- Auf der Startseite deines Repositorys zeigt GitHub die formatierte README an.

### Typische Fehler

- **Markdown erscheint als Rohtext:** Prüfe, ob die Datei `README.md` heißt und die Markdown-Zeichen korrekt gesetzt sind.
- **Ein Link führt ins Leere:** Vergleiche den Pfad im Link mit Dateiname und Ordnerstruktur.
- **Ein Codeblock bleibt offen:** Prüfe, ob auf drei öffnende Backticks auch drei schließende folgen.
- **Eine Tabelle ist schwer zu lesen:** Ersetze sie durch Überschriften und Listen, besonders wenn Inhalte Pipe-Zeichen enthalten.
- **Die README verspricht etwas, das nicht funktioniert:** Entferne oder korrigiere ungeprüfte Schritte, Links und Aussagen.

## Weiterlernen

Die GitHub-Dokumentation erklärt die Formatierungsmöglichkeiten. Die GFM-Spezifikation und der Markdown Guide bieten weiterführende Referenzen.[5]

## Glossar

- **Backtick:** Zeichen `` ` ``. Begrenzt kurze Code- oder Dateinamen im Fließtext.
- **Commit:** gespeicherter Änderungssatz in einem Git-Repository.
- **GitHub Flavored Markdown (GFM):** Markdown-Variante, die GitHub für Inhalte auf seiner Plattform unterstützt.
- **Markdown:** einfache Auszeichnungssprache für strukturierten Text.
- **Pipe:** senkrechter Strich (`|`); trennt beispielsweise Spalten in Markdown-Tabellen.
- **README:** Einstiegs- und Informationsdatei eines Projekts, häufig `README.md`.
- **Relativer Link:** Link, dessen Zielpfad relativ zur aktuellen Datei angegeben wird.
- **Repository:** Projektbereich mit Dateien und – bei Git – einer Versionsgeschichte.

## Schlagworte

`Markdown README GitHub Git Dokumentation Lern-Repository`

[5] GitHub-Dokumentation „Writing on GitHub“, GFM-Spezifikation 0.29-gfm (2019-04-06) und Markdown Guide „Basic Syntax“. Geprüft am 7. November 2026: https://docs.github.com/en/get-started/writing-on-github[6], https://github.github.com/gfm/[7], https://www.markdownguide.org/basic-syntax/[8]